

מבני נתונים – פרויקט מספר 2 – ערמות מתקדמות

הקדמה

בתרגיל זה שני חלקים:

- חלק מעשי: מימוש של ערימת פיבונאצ'י עם מספר תוספות. עמודים 1-3 במסמך מתארים חלק זה.
- חלק ניסוי-תאורטי: בהתבסס על המימוש מהחלק המעשי, נבצע מספר ניסויים עם ניתוח תאורטי נלווה. עמודים 4-7 מתאר חלק זה.

שימו לב: בסוף המסמך (עמוד 7) ישנן הוראות הגשה – הקפידו לפעול לפיהן. **תאריך הגשה: 25/01/2026**

חלק מעשי

דרישות

בתרגיל זה יש לממש ערימת פיבונאצ'י במימוש כפי שלמדנו בכיתה אבל עם מספר תוספות ושינויים. לכל איבר במבנה יש מפתח (key) שהוא מספר טבעי ומידע (info) שהוא מחרוזת. **המפתחות לא בהכרח ייחודיים, וכלל הערמה מתייחס כרגיל אך ורק למפתחות.** המימוש יהיה בשפת **Java 21** וצריך להיות מבוסס על קובץ השלד המופיע באתר הקורס.

הבנאי של המחלקה מקבל שני משתנים בוליאניים: lazyMelds ו lazyDecreaseKeys. שני המשתנים האלו נשארים זהים לכל אורך חיי מבנה הנתונים ולא משנים את ערכם. שני המשתנים הבוליאניים האלו קובעים את ההבדלים בין ההתנהגות של המבנה לבין ערימת פיבונאצ'י, כפי שלמדנו בכיתה (כלומר למחלקה שתממשו ייתכנו ארבע התנהגויות שונות בהתאם לערכים האפשריים של שני המשתנים הנ"ל). ניתן להניח שאם מתבצעת פעולת meld בין שתי ערימות אז לשתייהן יש אותו ערך של lazyMelds ואותו ערך של lazyDecreaseKeys.

כעת נגדיר את ההשפעה של lazyMelds ו lazyDecreaseKeys על ההתנהגות של מבנה הנתונים.

בפעולת meld משרשרים את רשימות העצים של שתי הערימות לרשימה אחת (כפי שלמדנו בכיתה בערימות פיבונאצ'י). לאחר מכן, אם lazyMelds=false אז (בשונה מערימת פיבונאצ'י) יש לבצע successive linking על רשימת העצים. אחרת (lazyMelds=true) לא מבצעים successive linking, כמו בערימת פיבונאצ'י.

בלי שום קשר לאמור לעיל, זכרו ש successive linking זו פעולה שאנו מבצעים בכל מקרה ב deleteMin, כמו בערימת פיבונאצ'י.

- כאשר מבצעים פעולת decreaseKey: אם המפתח החדש של הצומת (לאחר ההקטנה) גדול או שווה למפתח של אביו אז אין צורך לבצע פעולות נוספות כי כלל הערימה לא מופר. אחרת יש לפעול כדלקמן:
- אם lazyDecreaseKeys=true אז פועלים כמו בערימת פיבונאצ'י: מבצעים cascading cut (כולל התחזוק והטיפול בסימונים של הצמתים, כפי שלמדנו בכיתה) ומוסיפים את הצומת שנותק לרשימת השורשים. את ההוספה הזו של תת העץ שנותק לרשימת השורשים יש לבצע באמצעות meld בין תת העץ שנותק לשאר הערימה. שימו לב שאת meld אנו מבצעים עבור כל צומת שנותק במסגרת cascading cut, כלומר אם יש מסלול ארוך של צמתים מסומנים שאותו ניתקנו אז יבוצעו הרבה פעולות meld במסגרת cascading cut. כאמור, אם lazyMelds=false אז בניגוד לערימת פיבונאצ'י, כל אחד מהmeld-ים האלו יגרום לביצוע successive linking.
 - אחרת (lazyDecreaseKeys=false) נבצע heapifyUp כמו ב decreaseKey של ערימות בינומיות רגילות וערימות בינומיות עצלות.

שימו לב שכאשר `lazyDecreaseKeys=false` ו `lazyMelds=false` אז המבנה מתנהג בצורה דומה מאוד לערימה בינומית רגילה, אם `lazyDecreaseKeys=false` ו `lazyMelds=true` אז הערימה מתנהגת כמו ערימה בינומית עצלה ואם `lazyDecreaseKeys=true` ו `lazyMelds=true` אז המבנה מתנהג כמו ערימת פיבונאצ'י. את המימוש כאשר `lazyDecreaseKeys=true` ו `lazyMelds=false` לא למדנו בכיתה.

שימו לב שבכל מקרה, כפי שלמדנו בכיתה, ללא תלות בערכם של `lazyMelds` ו `lazyDecreaseKeys`: יש לממש את `delete` באמצעות `decreaseKey` ו `deleteMin`. כמו כן `insert` ו `deleteMin` ימומשו באמצעות `.meld`.

הפעולות שיש לממש הן:

פְּעוּלָה	תִּיאוֹר	
Heap(lazyMelds, lazyDecreaseKeys)	הבנאי של המחלקה יקבל כפרמטר את הפרמטרים הבוליאניים <code>lazyMelds</code> ו <code>lazyDecreaseKeys</code> שתוארו לעיל.	1
insert(k, info)	הכנסת איבר בעל מפתח <code>k</code> לערימה עם מידע <code>info</code> . הפונקציה מחזירה מצביע לצומת בערימה שנוצר עבורו.	2
findMin()	הפונקציה מחזירה את איבר הערימה בעל המפתח המינימלי.	3
deleteMin()	מחיקת האיבר המינימלי מהערימה (אין צורך להחזיר).	4
decreaseKey(x, d)	הפונקציה מקבלת מצביע לאיבר הערימה <code>x</code> וטבעי <code>d</code> . היא מפחיתה את המפתח של <code>x</code> ב- <code>d</code> ומתקנת את הערימה בהתאם לערך של <code>lazyDecreaseKeys</code> , כפי שהוסבר לעיל.	5
delete(x)	הפונקציה מקבלת מצביע לאיבר הערימה <code>x</code> ומוחקת אותו מהמבנה.	6
meld(heap2)	הפונקציה ממזגת את הערימה עם ערימה נוספת <code>heap2</code> . ניתן להניח של <code>heap2</code> יש ערכי <code>lazyDecreaseKeys</code> ו <code>lazyMelds</code> שזהים לערכים של המשתנים האלו באובייקט הנוכחי (<code>this</code>). יש לממש את הפעולה בהתאם לערך של <code>lazyMelds</code> כפי שתואר לעיל. לאחר הקריאה לפעולה הערימה <code>heap2</code> אינה שמישה.	7
size()	הפונקציה מחזירה את מספר האיברים בערימה.	8
numTrees()	הפונקציה מחזירה את מספר העצים בערימה.	9
numMarkedNodes()	הפונקציה מחזירה את מספר הצמתים המסומנים בערימה. שימו לב שאם <code>lazyDecreaseKeys=false</code> אז לא ייתכן שבערימה יש צמתים מסומנים.	10
totalLinks()	הפונקציה מחזירה את סך החיבורים (<code>links</code>) המצטבר שבוצעו (חיבור שני עצים מאותה הדרגה) מרגע יצירת הערימה.	11
totalCuts()	הפונקציה מחזירה את סך החיתוכים (ניתוק קשת) המצטבר שבוצעו מרגע יצירת הערימה, כלומר את מספר הפעמים שבהם צומת התנתק מאביו (הוצא מרשימת הבנים שלו והוסף לרשימת השורשים) שבוצעו (במסגרת <code>cascading cuts</code> , כולל החיתוך הראשוני של הצומת <code>x</code> , אם קרה). שימו לב שניתוק של צומת מאביו יכול לקרות כאשר קוראים ל <code>decreaseKey</code> אבל גם כאשר קוראים ל <code>delete</code> . כמו כן, ניתוק יכול להתבצע רק אם <code>lazyDecreaseKeys=true</code> . כאשר אנו מבצעים <code>deleteMin</code> אנחנו מנתקים את הילדים של המינימום ומכניסים אותם לרשימת השורשים: במקרה הזה לא נוסיף את החיתוכים האלו למספר החיתוכים הכללי של הערימה.	12
totalHeapifyCosts()	הפונקציה מחזירה את סך העלויות של ביצוע פעולות <code>heapifyUp</code> שבוצעו מרגע יצירת הערימה. שימו לב שאנו מבצעים <code>heapifyUp</code> רק במקרה שבו <code>lazyDecreaseKeys=false</code> זה קורה כאשר קוראים ל <code>decreaseKey</code> או <code>delete</code> . אנו מגדירים את העלות של פעולת <code>heapifyUp</code> (בודדת) על צומת <code>v</code> להיות מספר הפעמים שהחלפנו בין <code>v</code> לבין אביו. במילים אחרות מדובר בהפרש בין הרמה של <code>v</code> בעץ לפני ביצוע הפעולה לרמה של <code>v</code> בעץ אחרי ביצוע הפעולה.	13

הערה: כאשר מבצעים `meld` אז נדרש להוסיף את היסטוריית ה `links`, ה `cuts` וה `heapifyCosts` של `heap2` לערימה שאליה מבצעים את המעשה `meld` כי המשמעות של הפעולות `totalLinks`, `totalCuts` ו `totalHeapifyCosts` היא שמירה של תכונות של ההיסטוריה של הערימה. ב `meld` אנו מאחדים שתי ערימות לערימה אחת (ולכן רוצים לשמור את התכונות של ההיסטוריה ה"מאוחדת" שלה).

לצורך מימוש פעולות אלו, ניעזר במחלקה **HeapNode** המופיעה בקובץ.
המחלקה HeapNode המייצגת צומת בערמה מכילה את השדות הבאים:

- key – המפתח ששמור בצומת זה.
- info – המידע ששמור בצומת זה.
- child – בן כלשהוא של צומת זה.
- next – האח הבא של צומת זה.
- prev – האח הקודם של צומת זה.
- parent – ההורה של צומת זה בערמה.
- rank – דרגת הצומת (מספר הבנים).

הערות חשובות:

1. המימוש יבוצע על ידי מילוי קובץ השלד. מותר להחליף את תוכן הפונקציות הקיימות ולהוסיף פונקציות ושדות חדשים. אסור לשנות את חתימות הפונקציות הקיימות ואת שמות השדות הקיימים כדי לא לפגוע בטסטר. על כל הפונקציות/מחלקות להופיע בקובץ יחיד.
2. אין להשתמש באף מימוש ספרייה של מבנה נתונים.
3. עליכם לממש את כל הפעולות בסיבוכיות המיטבית.

סיבוכיות

יש לציין בקוד ולהסביר בקצרה במסמך התיעוד את סיבוכיות זמן הריצה במקרה הגרוע (האסימפטוטית, במונחי O הדוקים) של כל פונקציה שמכילה לולאות/רקורסיה, כתלות במספר האיברים בערימה n.

פלט

אין צורך בפלט למשתמש.

תיעוד

בנוסף לבדיקות אוטומטיות של הקוד שיוגש, קובץ המקור ייבדק גם באופן ידני. חשוב להקפיד על תיעוד לכל פונקציה וכמות סבירה של הערות. הקוד צריך להיות קריא, בפרט הקפידו על בחירת שמות משתנים ועל אורך השורות.

יש להגיש בנוסף לקוד גם מסמך תיעוד חיצוני. המסמך יכלול את תיאור תמציתי של המחלקה שמומשה, ואת תפקידו של כל חבר במחלקה. עבור כל פונקציה במחלקה יש להסביר מה היא עושה, כיצד היא פועלת ומהי סיבוכיות זמן הריצה שלה. בפרט, אם פונקציה קוראת לפונקציית עזר, יש להתייחס גם לפונקציית העזר בניתוח. עבור פונקציות שעולות זמן קבוע יספיק תיאור קצר ולא לפרט את ניתוח הסיבוכיות.

בדיקות

התרגילים ייבדקו באמצעות תוכנת טסטר שקוראת לפונקציות המפורטות מעלה בתרחישים שונים, ומוודאת את נכונות התוצאות. מומלץ מאוד לממש אוסף בדיקות עבור המימוש, לא בשביל ההגשה, אלא כדי לבדוק שהקוד לא רק רץ, אלא גם נכון!

בקובץ שתגישו לא תהיה פונקציית **main** ולא יהיו הרצות קוד/הדפסות, דבר זה יפגע בטסטר שיבדוק לכם את התרגילים. אין צורך להגיש את הקוד הנוסף שכתבתם לחלק הניסויי.

חלק ניסויי/תאורטי

את הערימה שמתקבלת כאשר lazyDecreaseKeys=true ו lazyMelds=false (שאותה כאמור לא למדנו בכיתה) נכנה "ערימה בינומית עם ניתוקים". כאמור, בערימה הזו מבצעים successive linking בכל meld (לכן בפרט גם בכל insert) אבל גם לאחר כל ניתוק צומת במסגרת תהליך cascading cutn בפעולה decreaseKey. שימו לב ש successive linking זו פעולה **שתמיד** רצה בזמן $\Omega(\log n)$ כי בכל מקרה בסוף צריך לעבור על מערך ה"סלים" שאורכו לוגריתמי, לקחת מכל תא לא ריק את העץ ששמור בו ולצרף את העצים לרשימת שורשים אחת.

1. בצעו ניתוח זמן ריצה במקרה הגרוע לזמני הריצה של הפעולות insert, findMin, deleteMin, decreaseKey ו delete בערימה בינומית עם ניתוקים עבור ערימה בגודל n .

2. בצעו ניתוח לשיעורין לזמני הריצה של הפעולות הנ"ל בערימה בינומית עם ניתוקים. הניחו שלכל אורך סדרת הפעולות מובטח שמספר האיברים בערימה בכל רגע הוא לכל היותר n .

נתבונן במימוש כאשר lazyDecreaseKeys=false ו lazyMelds=false. המימוש הזה מתנהג כמו ערימה בינומית אבל הביצועים שלו לא זהים לערימה בינומית: בערימה בינומית נהוג לשמור את העצים ברשימת השורשים מסודרים בסדר הדרגות ואת meld ומבצעים באופן דומה לחיבור בינארי של שני מספרים. לכן ייתכן שזמן הריצה של פעולת meld בודדת (או של פעולת insert בודדת) יהיה נמוך בהרבה מ $\log$. בפרט, בערימה בינומית פעולת insert מתבצעת באופן דומה לפעולת increment במונה בינארי וסדרה של n הכנסות לערימה בינומית ריקה תרוץ בזמן $O(n)$, בגלל אותם השיקולים של ניתוח אמורטייזד שדנו בהם בתרגול בנייתוח של המונה הבינארי. לעומת זאת במימוש במטלה כאשר lazyMelds=true ו lazyDecreaseKeys=true משתמשים ב successive linking (זו פעולה שהצגנו בקורס רק בשלב שבו דנו בערימות בינומיות עצלות) וכאמור, הפעולה הזו רצה **תמיד** בזמן $\Omega(\log n)$. על מנת לוודא שהביצועים של המימוש כאשר lazyDecreaseKeys=true ו lazyMelds=true יהיו זהים אסימפטוטית לערימה בינומית, ניתן לבצע עבודה אלטרנטיבית (במקום הקריאה ל successive linking). מכיוון שרצינו לחסוך מכם את הביצוע של העבודה הזו, הנחנו אתכם להשתמש ב successive linking במימוש של meld כאשר רוצים לממש ערימה בינארית. עם זאת, בנייתוח התיאורטי שתמלאו על ערימה בינומית בסעיפים 3 ו 4 - **עליכם למלא את התשובות על ערימה בינומית כפי שלמדנו בכיתה**, אילו היינו ממשים אותה במטלה הזו כפי שצריך, ללא שימוש ב successive linking.

3. באמצעות סעיפים 1 ו 2 - מלאו את שתי הטבלאות הבאות. בטבלה הראשונה מלאו את זמני הריצה במקרה הגרוע ובטבלה השנייה מלאו את זמני הריצה לשיעורין (כפונקציה של n).

זמני הריצה במקרה הגרוע:

<u>פעולה</u>	<u>ערימה בינומית</u>	<u>ערימה בינומית עצלה</u>	<u>ערימת פיבונאצ'י</u>	<u>ערימה בינומית עם ניתוקים</u>
insert				
findMin				
deleteMin				
decreaseKey				
delete				

זמני הריצה באמורטיזד:

פעולה	ערימה בינומית	ערימה בינומית עצלה	ערימת פיבונאצ'י	ערימה בינומית עם ניתוקים
insert				
findMin				
deleteMin				
decreaseKey				
delete				

בחלק זה נערוך שלושה ניסויים המשתמשים בארבעת סוגי הערימות המתקדמות: ערימה בינומית, ערימה בינומית עצלה, ערימת פיבונאצ'י וערימה בינומית עם ניתוקים. מזכיר שוב שבמימוש שעשיתם אפשר "לקבל" כל אחת מהערימות האלו באמצעות מתן ערכים מתאימים לlazyMelds ול lazyDecreaseKeys. עבור כל ניסוי נמדוד את זמן הריצה, מספר החיבורים, מספר החיתוכים, סה"כ העלויות של heapify Up, מספר העצים בסיום והעלות המקסימלית של פעולה בודדת. בכל הניסויים נגדיר $n = 464,646$. לצורך הניסויים, בנו מערך מצביעים המחזיק באינדקס i מצביע לאיבר בעל מפתח i .

- ניסוי ראשון:
 - נכניס לערמה ריקה את האיברים $1, \dots, n$ בסדר אקראי.
 - נמחק את המינימום.
- ניסוי שני:
 - נכניס לערמה ריקה את האיברים $1, \dots, n$ בסדר אקראי.
 - נמחק את המינימום.
 - נמחק את המקסימום (בעזרת מצביע) עד שנישאר עם 46 איברים.
- ניסוי שלישי:
 - נכניס לערמה ריקה את האיברים $1, \dots, n$ בסדר אקראי.
 - נמחק את המינימום.
 - נבצע $\lfloor 0.1n \rfloor$ פעמים הקטנת מפתח של המקסימום (בעזרת מצביע) 0 (כלומר עד שנישאר עם 90% איברים שהמפתח שלהם חיובי וכל שאר האיברים הם עם מפתח 0).
 - נבצע מחיקת מינימום שוב.

4. באמצעות מה שעניתם בסעיף 3 - מלאו את הטבלה הבאה. בכל תא יש לרשום את זמן הריצה האסימפטוטי של הניסוי המתאים על סוג הערימה המתאים כפונקציה של n . כמו בסעיף 3, עליכם למלא את התשובות על ערימה בינומית במימוש כפי שלמדנו בכיתה, אילו היינו מממשים אותה במטלה הזו כפי שצריך, ללא שימוש ב successive linking.

	ערימה בינומית	ערימה בינומית עצלה	ערימת פיבונאצ'י	ערימה בינומית עם ניתוקים
ניסוי 1				
ניסוי 2				
ניסוי 3				

5. עבור כל ניסוי מבין שלושת הניסויים שהוגדרו:

- הריצו אותו על כל אחת מארבעת הערימות המתקדמות.
- יש למלא בטבלה את הממוצע על פני 20 ניסויים זהים. לשם כך עבור כל ניסוי יש להגריל 20 פרמוטציות אקראיות של האיברים $1, \dots, n$, ועבור כל פרמוטציה שהוגרלה יש לבצע את הניסוי על כל אחד מארבעת סוגי הערימות עם הפרמוטציה שהוגרלה.
- בשורה "עלות מקסימלית לפעולה" יש לרשום את העלות המקסימלית של פעולה בודדת שבוצעה בסדרה (הכנסה / מחיקת מינימום / מחיקה / הקטנת מפתח). אנו מגדירים את העלות של פעולה בודדת להיות הסכום של מספר החיבורים, מספר החיתוכים ועלויות של heapify Up שבוצעו בפעולה.

ניסוי 1:

ערימה בינומית עם ניתוקים	ערימת פיבונאצ'י	ערימה בינומית עצלה	ערימה בינומית	
				זמן ריצה (מילישניות)
				גודל הערימה בסיום
				מספר העצים בסיום
				מספר חיבורים
				מספר חיתוכים
				סה"כ עלויות של heapify up
				עלות מקסימלית לפעולה

ניסוי 2:

ערימה בינומית עם ניתוקים	ערימת פיבונאצ'י	ערימה בינומית עצלה	ערימה בינומית	
				זמן ריצה (מילישניות)
				גודל הערימה בסיום
				מספר העצים בסיום
				מספר חיבורים
				מספר חיתוכים
				סה"כ עלויות של heapify up
				עלות מקסימלית לפעולה

ערימה בינומית עם ניתוקים	ערימת פיבונאצ'י	ערימה בינומית עצלה	ערימה בינומית	
				זמן ריצה (מילישניות)
				גודל הערימה בסיום
				מספר העצים בסיום
				מספר חיבורים
				מספר חיתוכים
				סה"כ עלויות של heapify up
				עלות מקסימלית לפעולה

6.

- א. האם יש הבדלים בביצועים של הניסויים בין הערימות השונות?
 ב. האם התוצאות שקיבלתם בסעיף 5 מאששות או מפריכות את מה שעניתם בסעיף 4?
 ג. האם בניסויים היו הבדלים בין העלות המקסימלית של פעולה לבין העלות הממוצעת של פעולה?

הוראות הגשה

הגשת התרגיל תתבצע באופן אלקטרוני באתר הקורס במודל.
 הגשת התרגיל היא בזוגות בלבד!
 כל זוג יבחר נציג/ה ויעלה רק תחת שם המשתמש של הנציג/ה את קבצי התרגיל (תחת קובץ **zip**) למודל.

על ההגשה לכלול שלושה קבצים:

1. קובץ המקור (הרחבה של קובץ השלד שניתן) תחת השם **Heap.java**.
2. קובץ טקסט **info.txt** המכיל את פרטי הזוג: שמות מלאים בעברית, שמות מלאים באנגלית ומספרי ת"ז. יש לרשום גם שמות משתמש (או לחילופין אימיילים אוניברסיטאיים) של המגישים.
3. מסמך בפורמט **pdf** שמכיל קודם את התיעוד החיצוני ולאחר מכן את המענה לחלק הניסויי/תיאורטי.

שמות קובץ התיעוד וקובץ הzip צריכים לכלול את שמות המשתמש האוניברסיטאיים של הזוג המגיש לפי הפורמט **Heap_username1_username2.pdf/zip**, בתוכן הקבצים יש לציין את שמות המשתמש, תעודות הזהות ושמות המגישים (בכותרת המסמך ובשורת הערה בקובץ המקור).

הגשת שיעורי הבית באיחור - באישור מראש בלבד. הגשה באיחור ללא אישור תגרור הורדת נקודות מהציון. הגשת התרגיל היא חובה לשם קבלת ציון בקורס.

בהצלחה!

Cascading cuts

Consolidating / Successive linking

Consolidating / Successive linking

```

Function cut(x, y)
  x.parent ← null
  x.mark ← 0
  y.rank ← y.rank - 1
  if x.next = x then
    | y.child ← null
  else
    | y.child ← x.next
    | x.next.prev ← x.prev

```

Cut x from its parent y

```

Function cascading-cut(x, y)
  cut(x, y)
  if y.parent ≠ null then
    if y.mark = 0 then
      | y.mark ← 1
    else
      | cascading-cut(y, y.parent)

```

Perform a cascading-cut process starting at x

```

Function consolidate(x)
  to-buckets(x)
  return from-buckets()

```

```

Function to-buckets(x)
  for i ← 0 to log2 n do
    | B[i] ← null
  x.prev.next ← null
  while x ≠ null do
    | y ← x
    | while B[y.rank] ≠ null do
      | y ← link(y, B[y.rank])
    | B[y.rank - 1] ≥ null
    | B[y.rank] ← y

```

```

Function from-buckets()
  x ← null
  for i ← 0 to log2 n do
    if B[i] ≠ null then
      if x = null then
        x ← B[i]
        x.next ← x
        x.prev ← x
      else
        insert-after(x, B[i])
        if B[i].key < x.key then
          x ← B[i]
  return x

```

115

```

Function consolidate(x)
  to-buckets(x)
  return from-buckets()

```

```

Function to-buckets(x)
  for i ← 0 to log2 n do
    | B[i] ← null
  x.prev.next ← null
  while x ≠ null do
    | y ← x
    | while B[y.rank] ≠ null do
      | y ← link(y, B[y.rank])
    | B[y.rank - 1] ≥ null
    | B[y.rank] ← y

```

```

Function from-buckets()
  x ← null
  for i ← 0 to log2 n do
    if B[i] ≠ null then
      if x = null then
        x ← B[i]
        x.next ← x
        x.prev ← x
      else
        insert-after(x, B[i])
        if B[i].key < x.key then
          x ← B[i]
  return x

```

115

אי פֿיבונאצ'י

Linking binomial trees

```

Function link(x, y)
  if x.key > y.key then
    | x ↔ y
  y.next ← x.child
  x.child ← y
  return x

```

Linking in first representation

```

Function link(x, y)
  if x.key > y.key then
    | x ↔ y
  if x.child = null then
    | y.next ← y
  else
    | y.next ← x.child.next
    | x.child.next ← y
  x.child ← y
  return x

```

Linking in second representation

Linking binomial trees

```

Function link(x, y)
  if x.key > y.key then
    | x ↔ y
  y.next ← x.child
  x.child ← y
  return x

```

אינסערציע
Linking in first representation

```

Function link(x, y)
  if x.key > y.key then
    | x ↔ y
  if x.child = null then
    | y.next ← y
  else
    | y.next ← x.child.next
    | x.child.next ← y
  x.child ← y
  return x

```

אינסערציע
Linking in second representation

Heapsort Complexity

- Input: array with n elements
- Output: array sorted
- Algorithm:
 - Create min-heap from input $O(n)$
 - Do delete-min, and put the deleted element at the last position of the array. Repeat n times. $O(n \log n)$
 - Reverse the array (or use a max-heap instead) $O(n)$
- Overall time complexity $O(n \log n)$, additional memory $O(1)$

Application of Heaps

- Input: array with n elements, index k
- Output: The k smallest elements in the array
- Algorithm:
 - Create min-heap from input $O(n)$
 - Do delete-min, and put the deleted element at the last position of the array. Repeat k times. $O(k \log n)$
- Overall time complexity $O(n + k \log n)$

Delete-min Heapify-Down

```

Function Heapify-down(Q, i)
  1 l ← left(i)
  2 r ← right(i)
  3 smallest ← i
  4 if l < size(Q) and Q[l] < Q[smallest]
  5 | then smallest ← l
  6 if r < size(Q) and Q[r] < Q[smallest]
  7 | then smallest ← r
  8 if smallest > i then
  9 | Q[i] ↔ Q[smallest]
  10 | Heapify-down(Q, smallest)

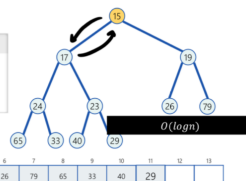
```

Inserting an Item Heapify-up

```

Function Heapify-up(Q, i)
  while i > 1 and Q[i] < Q[parent(i)] do
    | Q[i] ↔ Q[parent(i)]
    | i ← parent(i)

```



Application of Heaps

- Input: array with n elements, index k
- Output: The k smallest elements in the array
- Algorithm:
 - Create min-heap from input $O(n)$
 - Do delete-min, and put the deleted element at the last position of the array. Repeat k times. $O(k \log n)$
- Overall time complexity $O(n + k \log n)$

ערמת פיבונאצ'י

- נמשך Decrease-Key מהר יותר
- במקום לפעפע צומת כלפי מעלה - נחתוך אותו מהעץ הנוכחי, ונהפוך אותו לשורש
- בכל פעם שצומת (שאינו שורש) מאבד שני בנים - נחתוך גם אותו
 - למה לחתוך אותו?
 - לשם כך נחזיק סימון לכל צומת - צומת מסומן אם הוא איבד בן אחד
- התהליך נקרא cascading-cut
- שאר הפעולות כמו בערמה בינומית עזלה
- העצים הם כבר לא עצים בינומיים, אבל דרגות השורשים נשארות $O(\log n)$



פעולות על ערמה בינומית

- $\text{Insert}()$, $\text{Merge}()$ - כמו חיבור מספרים בינאריים
 - כמו בדוגמת המונה, n פעולות $\text{Insert}()$ המתחילות ממבנה ריק ידרשו $O(n)$ זמן
- $\text{Delete-Min}()$ - נמחק את הצומת שמכיל את האיבר, העץ הבינומי יתפרק לאוסף של עצים בינומיים, נמזג את האוסף הזה עם שאר הערמה
 - כל בן של השורש היה פעם שורש של עץ בינומי בעצמו
 - כלל הערמה מתקיים בכל העץ
- $\text{Decrease-Key}()$ - כמו בערמה בינארית, האיבר יפעפע למעלה